

**LAPORAN PRAKTIKUM**  
**ALGORITMA DAN PEMROGRAMAN 2**  
**MODUL 17**  
**“SKEMA PEMROSESAN SEKUENSIAL”**



**DISUSUN OLEH:**  
**NADIFA AZKHIA SYARIF**  
**109082530002**  
**S1 IF-13-02**

**DOSEN:**  
**Wahyu Andi Saputra, S.Pd., M.Eng**

**PROGRAM STUDI S1 TEKNIK INFORMATIKA**  
**FAKULTAS INFORMATIKA**  
**TELKOM UNIVERSITY PURWOKERTO**  
**2025/2026**

## DASAR TEORI

Menurut Modul 17 Praktikum Algoritma dan Pemrograman 2, skema pemrosesan sekuensial merupakan pola pengolahan data yang menggunakan struktur perulangan dan percabangan untuk menyelesaikan berbagai masalah komputasi. Pada modul dijelaskan bahwa “dengan dipersenjatai bentuk perulangan dan bentuk percabangan, banyak problem komputasi yang dapat diselesaikan.” Pernyataan tersebut menunjukkan bahwa penggunaan struktur kontrol sangat penting dalam proses pemrograman untuk mengolah data secara sistematis dan berurutan.

Menurut Sukanto dan Shalahuddin (2018), algoritma adalah urutan langkah-langkah logis yang digunakan untuk menyelesaikan suatu masalah secara sistematis. Dalam pemrograman, algoritma menjadi dasar utama dalam pembuatan program karena menentukan alur proses dan cara kerja program tersebut. Penggunaan algoritma yang baik dapat membuat program lebih efisien, terstruktur, dan mudah dipahami.

Menurut Rosa A.S dan M. Shalahuddin (2018), struktur perulangan digunakan untuk menjalankan suatu instruksi secara berulang hingga kondisi tertentu terpenuhi. Struktur perulangan sangat penting dalam pemrosesan data sekuensial karena memungkinkan program membaca dan memproses data dalam jumlah banyak secara otomatis tanpa harus menuliskan kode yang sama berulang kali.

Menurut Kadir (2020), percabangan merupakan struktur kontrol yang digunakan untuk menentukan keputusan berdasarkan kondisi tertentu. Dengan adanya percabangan, program dapat memilih proses yang berbeda sesuai kondisi yang terjadi. Konsep ini banyak digunakan pada pemrosesan data dengan marker maupun pada kondisi data kosong agar program dapat berjalan dengan benar.

Selain itu, penggunaan bilangan acak dalam pemrograman digunakan untuk simulasi dan pemodelan suatu kejadian. Menurut Munir (2019), bilangan acak sering dimanfaatkan dalam metode simulasi, salah satunya metode Monte Carlo yang digunakan untuk memperkirakan nilai matematika seperti konstanta  $\pi$  melalui percobaan acak. Konsep ini diterapkan pada simulasi topping pizza dan curah hujan dengan memanfaatkan koordinat acak menggunakan fungsi `rand.Float64()`.

## SOAL LATIHAN

### Statement skema

1.

### Source Code:

```
package main

import "fmt"

func main() {
    var bil, total float64
    var jumlah int

    fmt.Print("Masukkan bilangan: ")
    fmt.Scan(&bil)

    for bil != 9999 {
        total += bil
        jumlah++

        fmt.Print("Masukkan bilangan: ")
        fmt.Scan(&bil)
    }

    if jumlah > 0 {
        fmt.Println("Rerata =", total/float64(jumlah))
    } else {
        fmt.Println("Tidak ada data")
    }
}
```

### Output:

Masukkan bilangan: 10  
Masukkan bilangan: 20  
Masukkan bilangan: 30  
Masukkan bilangan: 9999  
Rerata = 20

### Deskripsi Program:

Program ini digunakan untuk menghitung nilai rerata dari sejumlah bilangan real yang dimasukkan pengguna. Program akan terus membaca data hingga pengguna memasukkan angka 9999 sebagai marker atau penanda akhir data. Semua bilangan sebelum marker akan dijumlahkan dan dihitung banyak datanya, kemudian program menampilkan hasil rerata dari seluruh bilangan tersebut. Program ini menerapkan konsep perulangan dengan marker pada pemrosesan data sekuensial.

## 2.

### Source Code:

```
package main

import "fmt"

func main() {
    var x, data string
    var n int

    var ditemukan bool
    var posisi int
    var jumlah int

    fmt.Print("Masukkan string x: ")
    fmt.Scan(&x)

    fmt.Print("Masukkan jumlah data: ")
    fmt.Scan(&n)

    for i := 1; i <= n; i++ {
        fmt.Print("Data ke-", i, ": ")
        fmt.Scan(&data)

        if data == x {
            jumlah++

            if !ditemukan {
                ditemukan = true
                posisi = i
            }
        }
    }

    // a
    if ditemukan {
        fmt.Println("a. String ditemukan")
    } else {
        fmt.Println("a. String tidak ditemukan")
    }

    // b
    if ditemukan {
```

```

        fmt.Println("b. Posisi pertama ditemukan di indeks",
posisi)
    } else {
        fmt.Println("b. String tidak memiliki posisi")
    }

    // c
    fmt.Println("c. Jumlah string x =", jumlah)

    // d
    if jumlah >= 2 {
        fmt.Println("d. Ada sedikitnya dua string x")
    } else {
        fmt.Println("d. Tidak ada sedikitnya dua string x")
    }
}

```

#### Output:

Masukkan string x: apel

Masukkan jumlah data: 5

Data ke-1: mangga

Data ke-2: apel

Data ke-3: jeruk

Data ke-4: apel

Data ke-5: semangka

a. String ditemukan

b. Posisi pertama ditemukan di indeks 2

c. Jumlah string x = 2

d. Ada sedikitnya dua string x

- a. String x ada dalam kumpulan data string apabila saat dilakukan pengecekan pada setiap data, ditemukan data yang sama dengan string x. Program menggunakan variabel boolean ditemukan untuk menandai apakah string tersebut ada atau tidak.
- b. Posisi string x ditemukan dengan menyimpan urutan data ketika string yang dimasukkan sama dengan x. Posisi pertama yang cocok akan disimpan ke dalam variabel posisi
- c. Jumlah string x dalam kumpulan data dihitung menggunakan variabel jumlah. Setiap kali ditemukan data yang sama dengan x, maka nilai jumlah akan bertambah satu.
- d. Untuk mengetahui apakah terdapat sedikitnya dua string x, program memeriksa nilai variabel jumlah. Jika jumlah lebih besar atau sama dengan 2, maka berarti ada minimal dua string x dalam kumpulan data.

#### Deskripsi Program:

Program ini digunakan untuk mencari sebuah string tertentu dalam kumpulan data string. Program membaca string x sebagai data yang dicari, kemudian membaca

sejumlah n data string lainnya. Setelah proses pencarian dilakukan, program dapat menentukan apakah string x ditemukan atau tidak, posisi pertama kemunculan string x, jumlah kemunculan string tersebut, serta memeriksa apakah terdapat sedikitnya dua string yang sama dalam kumpulan data. Program ini memanfaatkan proses pencarian dan perulangan untuk memeriksa setiap data string yang dimasukkan pengguna.

### 3.

#### Source Code:

```
package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var x, y float64
    var A, B, C, D int

    fmt.Print("Jumlah tetesan: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x = rand.Float64()
        y = rand.Float64()

        if x < 0.5 && y < 0.5 {
            A++
        } else if x >= 0.5 && y < 0.5 {
            B++
        } else if x >= 0.5 && y >= 0.5 {
            C++
        } else {
            D++
        }
    }

    fmt.Println("Curah hujan daerah A:", float64(A)*0.0001,
        "milimeter")
}
```

```

        fmt.Println("Curah hujan daerah B:", float64(B)*0.0001,
"milimeter")
        fmt.Println("Curah hujan daerah C:", float64(C)*0.0001,
"milimeter")
        fmt.Println("Curah hujan daerah D:", float64(D)*0.0001,
"milimeter")
    }

```

Output:

Jumlah tetesan: 1000000

Curah hujan daerah A: 25.0537000000000003 milimeter

Curah hujan daerah B: 24.9585 milimeter

Curah hujan daerah C: 25.0155000000000003 milimeter

Curah hujan daerah D: 24.9723 milimeter

Deskripsi Program:

Program ini digunakan untuk mensimulasikan pengukuran curah hujan pada empat daerah berbeda, yaitu daerah A, B, C, dan D. Program menerima input berupa jumlah tetesan hujan, kemudian membuat titik koordinat acak menggunakan fungsi `rand.Float64()`. Setiap titik yang dihasilkan akan dikelompokkan ke dalam salah satu daerah berdasarkan posisi koordinatnya. Setelah seluruh tetesan dihitung, program menampilkan total curah hujan pada masing-masing daerah dalam satuan milimeter. Program ini menggunakan konsep bilangan acak, percabangan, dan perulangan dalam proses perhitungannya

#### 4.

##### Source Code:

```

package main

import "fmt"

func main() {
    var n int
    var jumlah float64
    var suku float64

    fmt.Print("N suku pertama: ")
    fmt.Scan(&n)

    for i := 1; i <= n; i++ {

        if i%2 == 1 {
            suku = 1.0 / float64(2*i-1)
        } else {
            suku = -1.0 / float64(2*i-1)
        }
    }
}

```

```

        jumlah += suku
    }

    fmt.Println("Hasil PI:", 4*jumlah)
}

```

Output:

N suku pertama: 1000000

Hasil PI: 3.1415916535897743

**Source Code: berhenti jika selisih dua suku  $\leq 0.00001$**

```

package main

import (
    "fmt"
)

func main() {
    var jumlah float64
    var suku, sukuNext float64
    var i int = 1

    for {
        if i%2 == 1 {
            suku = 1.0 / float64(2*i-1)
        } else {
            suku = -1.0 / float64(2*i-1)
        }

        if (i+1)%2 == 1 {
            sukuNext = 1.0 / float64(2*(i+1)-1)
        } else {
            sukuNext = -1.0 / float64(2*(i+1)-1)
        }

        jumlah += suku

        selisih := suku - sukuNext
        if selisih < 0 {
            selisih = -selisih
        }

        if selisih <= 0.00001 {
            break
        }
    }
}

```

```

        i++
    }

    fmt.Println("Hasil PI:", 4*jumlah)
    fmt.Println("Pada i ke:", i)
}

```

Output:

Hasil PI: 3.1416026534897203

Pada i ke: 100001

Deskripsi Program:

Program ini digunakan untuk menghitung pendekatan nilai konstanta  $\pi$  menggunakan deret Leibniz. Program menghitung nilai  $\pi$  berdasarkan banyaknya suku yang dimasukkan pengguna dengan menjumlahkan deret harmonik ganti secara berulang. Setelah itu, program dimodifikasi agar proses perhitungan berhenti secara otomatis ketika selisih dua suku yang berurutan bernilai kurang dari atau sama dengan 0.00001. Program ini bertujuan untuk memahami penggunaan perulangan, operasi matematika, dan kondisi berhenti dalam suatu proses komputasi numerik.

## 5.

### Source Code:

```

package main

import (
    "fmt"
    "math/rand"
)

func main() {
    var n int
    var x, y float64
    var dalam int
    var pi float64

    fmt.Print("Banyak topping: ")
    fmt.Scan(&n)

    for i := 0; i < n; i++ {
        x = rand.Float64()
        y = rand.Float64()
    }
}

```

```
        if (x-0.5)*(x-0.5)+(y-0.5)*(y-0.5) <= 0.25 {
            dalam++
        }
    }

    pi = 4 * float64(dalam) / float64(n)

    fmt.Println("Topping pada Pizza:", dalam)
    fmt.Printf("PI : %.10f\n", pi)
}
```

#### Output:

Banyak topping: 3000

Topping pada Pizza: 2387

PI : 3.1826666667

#### Deskripsi Program:

Program ini digunakan untuk mensimulasikan penaburan topping pizza guna memperkirakan nilai konstanta  $\pi$ . Program membuat titik acak pada sebuah bidang persegi yang berisi lingkaran pizza. Setiap titik akan diperiksa apakah berada di dalam lingkaran pizza atau di luar lingkaran. Jumlah titik yang berada di dalam lingkaran dibandingkan dengan jumlah seluruh titik untuk menghitung pendekatan nilai  $\pi$  menggunakan metode Monte Carlo. Program ini menerapkan konsep geometri lingkaran, bilangan acak, perulangan, dan simulasi matematika dalam proses perhitungannya.

## DAFTAR PUSTAKA

Kadir, A. 2020. *Dasar Pemrograman*. Yogyakarta: Andi.

Modul Praktikum Algoritma dan Pemrograman 2. Modul 17 Skema Pemrosesan Sekuensial. Halaman 115-122.

Munir, R. 2019. *Algoritma dan Pemrograman dalam Bahasa Pascal dan C*. Bandung: Informatika.

Rosa A.S., dan M. Shalahuddin. 2018. *Rekayasa Perangkat Lunak Terstruktur dan Berorientasi Objek*. Bandung: Informatika.

Sukanto, R. A., dan M. Shalahuddin. 2018. *Logika Algoritma dan Pemrograman Dasar*. Bandung: Informatika.